# Лабораторная работа № 5

**Тема:** Введение в Prisma ORM 7: подключение к существующей PostgreSQL-базе

**Цель:** Настроить Prisma ORM 7 для существующего проекта shop-api, получить модели Product и Category из shop_db и проверить чтение данных с помощью Prisma Client, не изменяя REST API.

**Дисциплина:** РО 4.3. Интеграция серверных приложений с реляционными базами данных

---

## Лабораторная работа № 5. Введение в Prisma ORM 7

Тема: Подключение Prisma ORM 7 к существующей базе PostgreSQL shop_db в проекте shop-api. Работа выполняется на JavaScript (ES Modules) с Node.js 22 и Express 5.

Цель: освоить модель Prisma и introspection, настроить Prisma Client и подтвердить подключение к существующим таблицам products и categories двумя тестовыми запросами чтения.

> **Важно**
>
> В этой работе не переписываем productService.js, Controller и Router, не выполняем POST/PATCH/DELETE и не изменяем таблицы с помощью Prisma. Полный CRUD — тема лабораторной работы № 6.

## 1. Проверка исходной базы

Используйте уже созданную базу shop_db. В таблице products поле category удалено, category_id имеет NOT NULL; в categories имя категории хранится в поле name. Названия SQL-таблиц: products и categories.

pgAdmin Query Tool — проверить две таблицы и связь

```sql
SELECT id, name FROM categories ORDER BY id;

SELECT p.id, p.name, p.price, p.in_stock, p.category_id, c.name AS category_name
FROM products AS p
JOIN categories AS c ON c.id = p.category_id
ORDER BY p.id;
```

Сохраните результаты SELECT для отчёта. Если поле category_id ссылается на отсутствующую категорию, проверьте данные и ограничения в PostgreSQL.

## 2. Установка зависимостей Prisma 7

PowerShell — каталог shop-api

```
npm install -D prisma@7 typescript tsx @types/node @types/pg
npm install @prisma/client@7 @prisma/adapter-pg pg dotenv
```

prisma — CLI; @prisma/client — клиент моделей; @prisma/adapter-pg — адаптер PostgreSQL; tsx позволяет JavaScript-программе импортировать сгенерированные TypeScript-модули.

## 3. Инициализация Prisma

PowerShell — каталог shop-api

```
npx prisma init --datasource-provider postgresql --output ../generated/prisma
```

После выполнения команды найдите prisma/schema.prisma и файл конфигурации Prisma CLI в корне shop-api. Редактируйте именно имя конфигурационного файла, которое создала команда init (например, prisma.config.ts).

Структура файлов, важная для задания

```
shop-api/
├── config/
│   ├── db.js                # старый pg — пока не удаляем
│   └── prisma.js            # создадим
├── prisma/
│   └── schema.prisma
├── generated/
│   └── prisma/             # появится после generate
├── prisma.config.ts         # либо фактически созданное имя CLI-файла
├── .env
├── app.js
├── package.json
└── test-prisma.js           # создадим
```

## 4. Подключение CLI к PostgreSQL

Файл shop-api/.env — пример учебного подключения

```
DATABASE_URL="postgresql://shop_user:college123@localhost:5432/shop_db?schema=public"
```

**Важно**

Подставьте собственные параметры подключения, если они отличаются. Пароль из примера подходит только для учебной локальной базы. Не включайте .env в Git.

Файл Prisma CLI в корне проекта: prisma.config.ts (или имя после init)

```
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: { path: "prisma/migrations" },
  datasource: { url: process.env.DATABASE_URL }
});
```

Конфигурационный файл Prisma CLI используется командами db pull и generate. Express не импортирует его напрямую.

## 5. Получение моделей существующей базы

PowerShell — каталог shop-api

```
npx prisma db pull
```

Prisma считывает метаданные PostgreSQL: таблицы, поля, ключи, типы и связи. Команда не создаёт заново таблицы и не удаляет данные.

Проверьте, что из таблиц products и categories появились две модели. При introspection они могут называться products и categories — это нормально до переименования.

## 6. Настройка Prisma Schema

Файл: shop-api/prisma/schema.prisma. Оставьте блок datasource из результата и приведите generator к следующему виду:

shop-api/prisma/schema.prisma — generator client

```
generator client {
  provider             = "prisma-client"
  output               = "../generated/prisma"
  runtime              = "nodejs"
  moduleFormat         = "esm"
  importFileExtension = "ts"
}
```

Следующие модели соответствуют принятой структуре учебной БД. Изменения имён выполняются только в Prisma Schema: сами SQL-таблицы не переименовываются. Если реальные ограничения БД отличаются, сохраняйте определения db pull.

shop-api/prisma/schema.prisma — модели Product и Category

```
model Category {
  id       Int       @id
  name     String    @db.VarChar(100)
  products Product[]

  @@map("categories")
}

model Product {
  id         Int      @id @default(autoincrement())
  name       String   @db.VarChar(150)
  price      Int
  inStock    Boolean  @default(true) @map("in_stock")
  categoryId Int      @map("category_id")
  category   Category @relation(fields: [categoryId], references: [id], onDelete:
NoAction, onUpdate: NoAction, map: "fk_product_categorie")

  @@map("products")
}
```

Названия Product и Category — имена моделей в коде; @@map связывает их с таблицами products и categories. @map связывает поля JavaScript inStock и categoryId со столбцами in_stock и category_id. @relation задаёт связь Product с Category.

Если после db pull присутствуют дополнительные модели других таблиц, оставьте их без изменений; код для них не требуется.

## 7. Проверка и генерация Prisma Client

PowerShell — каталог shop-api

```
npx prisma validate
npx prisma generate
```

Команда validate проверяет синтаксис и связи Prisma Schema. generate создаёт клиент в generated/prisma. При ошибке проверьте имена моделей, relation-поля и фактическую структуру базы.

## 8. Создание подключения для Node.js

shop-api/config/prisma.js — новый файл

```
import "dotenv/config";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client.ts";

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
});

export const prisma = new PrismaClient({ adapter });
```

Файл config/prisma.js экспортирует один экземпляр Prisma Client. Обратите внимание на расширение .ts в пути client.ts — запуск программы выполняется с помощью tsx.

## 9. Тест чтения таблицы products

shop-api/test-prisma.js — тестовый скрипт

```
import { prisma } from "./config/prisma.js";

try {
  const products = await prisma.product.findMany();
  console.log("Products:", products);
} catch (error) {
  console.error("Ошибка Prisma:", error);
} finally {
  await prisma.$disconnect();
}
```

PowerShell — каталог shop-api

```
npx tsx test-prisma.js
```

В консоли должен появиться массив объектов Product. Если строк нет, будет []. Если возникает TypeError при вызове prisma.product, сопоставьте имя модели Product в schema.prisma и повторите prisma generate.

## 10. Тест чтения таблицы categories

shop-api/test-categories.js — тестовый скрипт

```
import { prisma } from "./config/prisma.js";

try {
  const categories = await prisma.category.findMany();
  console.log("Categories:", categories);
} finally {
  await prisma.$disconnect();
}
```

PowerShell — каталог shop-api

```
npx tsx test-categories.js
```

Убедитесь, что элементы массива содержат id и name. Эти два запроса нужны только для проверки подключения: подробный синтаксис CRUD разбирается на лекции № 6.

## 11. Дополнительная проверка через Prisma Studio

PowerShell — каталог shop-api

```
npx prisma studio
```

Откройте представление моделей в браузере и найдите Product и Category. Не удаляйте и не редактируйте записи: на этом занятии используем Prisma Studio только для просмотра.

## 12. Самостоятельное задание: другие таблицы

Для таблиц users, orders и order_items (если они присутствуют в вашей shop_db) определите, какие модели создала команда db pull. Код для этих моделей не приводится.

1. Выпишите название каждой SQL-таблицы и соответствующей Prisma-модели.
2. Найдите PRIMARY KEY и FOREIGN KEY в созданных моделях.
3. Объясните, какие поля могли бы участвовать в связях User → Order и Order → OrderItem.
4. При желании сделайте отдельный test-other-models.js с одной операцией чтения, используя фактическое имя модели из schema.prisma.

> **Важно**
>
> Не реализуйте пока create(), update(), delete() и не меняйте Express-сервисы для дополнительных таблиц. Это материал следующих занятий.

## 13. Требования к отчёту

5. Цель работы и краткое описание двух таблиц.
6. Скриншот проверки таблиц products и categories в pgAdmin.
7. Фрагмент schema.prisma с Product, Category и пояснением атрибутов @map, @@map, @relation.
8. Скриншот успешных prisma validate и prisma generate.
9. Содержимое config/prisma.js, test-prisma.js и test-categories.js.
10. Скриншоты двух успешных запусков npx tsx с результатами чтения.

11. Краткий вывод о роли Prisma ORM и результате подключения.

## 14. Контрольные вопросы

12. Что такое ORM?
13. Чем модель Prisma отличается от таблицы PostgreSQL?
14. Для чего предназначен файл prisma.config.ts (или созданный init аналог)?
15. Что делает prisma db pull?
16. Для чего используются @map и @@map?
17. Какую связь описывает categoryId и @relation?
18. Почему categoryId в Product объявлен как Int, а не Int?
19. Зачем запускать prisma generate?
20. Какую задачу выполняет config/prisma.js?
21. Почему тесты запускаются через tsx?

## 15. Критерии оценивания

**Обязательная часть — 100 баллов**

| Критерий | Баллы |
|---|---|
| Проверка PostgreSQL и исходных таблиц | 15 |
| Установка и конфигурация Prisma 7 | 20 |
| Introspection и корректные модели Product, Category | 25 |
| Генерация клиента и config/prisma.js | 15 |
| Успешные тесты чтения обеих таблиц | 15 |
| Отчёт и контрольные вопросы | 10 |
| ИТОГО | 100 |